Lecture 11 · Wed 11/4/2026

CS160 Compilers — Fall 2026

Operational Semantics, and Adding Lists.

Yu Feng

Two kinds of rules.

	Types (Lecture 9)	Values (today)
expression	$\Gamma \vdash e : T$	$S \vdash e : v, S'$
statement	$\Gamma, R \vdash s$	$S \vdash s : S'$

operational semantics
rules for how a program runs

S
the **store**: every variable's value, plus the output so far

v
the value

S'
the store afterwards

Γ maps each variable to its type. S maps each variable to its value.

Constants, variables, assignment.

$$\frac{}{S \vdash n : n, S}\ \text{E-Const} \qquad \frac{}{S \vdash x : S(x), S}\ \text{E-Var}$$

$$\frac{S \vdash e : v, S_1}{S \vdash x = e : S_1[x \mapsto v]}\ \text{S-Assign}$$

$S(x)$
the value of x in S

$S[x \mapsto v]$
S with x now holding v

A variable means its value in S. Assignment updates it. A declaration `x: T = e` uses the same rule.

Operators.

$$\frac{S \vdash e_1 : v_1, S_1 \qquad S_1 \vdash e_2 : v_2, S_2}{S \vdash e_1 \ \mathtt{op}\ e_2 : v_1 \ \mathtt{op}\ v_2, S_2}\ \text{E-BinOp} \quad \mathtt{op} \in \{+, -, *, <, <=, >, >=, ==, !=\}$$

subscripts

e_1, its value v_1, the store it leaves S_1

v_1 op v_2

computed as in arithmetic: 3 + 1 is 4, 4 < 5 is `True`

The second premise starts from S_1: the left operand runs first.

Statement lists and `return`.

$$\frac{S \vdash s_1 : S_1 \quad \langle ret \rangle \notin S_1 \quad S_1 \vdash s_2 \ \ldots \ s_n : S_2}{S \vdash s_1 \ s_2 \ \ldots \ s_n : S_2}\ \text{S-Seq}$$

$$\frac{S \vdash s_1 : S_1 \quad \langle ret \rangle \in S_1}{S \vdash s_1 \ s_2 \ \ldots \ s_n : S_1}\ \text{S-SeqReturn}$$

$$\frac{S \vdash e : v, S_1}{S \vdash \texttt{return}\ e : S_1[\langle ret \rangle \mapsto v]}\ \text{S-Return}$$

⟨ret⟩
a special variable no program can name: the value being returned

Statements run in order until one sets ⟨ret⟩. Then the rest are skipped.

Calling a function.

$$\frac{\texttt{def f}(x_1, \ldots, x_n)\texttt{: body} \quad \text{is in the program} \qquad S \vdash e_1 : v_1, S_1 \quad \ldots \quad S_{n-1} \vdash e_n : v_n, S_n \qquad S_n[x_1 \mapsto v_1, \ldots, x_n \mapsto v_n] \vdash \text{body} : S'}{S \vdash \texttt{f}(e_1, \ldots, e_n) : S'(\langle \texttt{ret} \rangle), S_n \triangleleft S'} \text{ E-Call}$$

$S_n \triangleleft S'$

after the call: the caller's variables come back; changes to lists and output stay

Evaluate the arguments, bind the parameters, run the body. The value is whatever `return` put in ⟨`ret`⟩.

if and while.

$$\frac{S \vdash c : \text{True}, S_1 \qquad S_1 \vdash s_1 \ldots s_n : S_2}{S \vdash \texttt{if c: } s_1 \ldots s_n \texttt{ else: } t_1 \ldots t_m : S_2}\ \text{S-IfTrue} \qquad (\text{S-IfFalse: run } t_1 \ldots t_m)$$

$$\frac{S \vdash c : \text{False}, S_1}{S \vdash \texttt{while c: s} : S_1}\ \text{S-WhileFalse}$$

$$\frac{S \vdash c : \text{True}, S_1 \qquad S_1 \vdash s : S_2 \qquad \langle \text{ret} \rangle \notin S_2 \qquad S_2 \vdash \texttt{while c: s} : S_3}{S \vdash \texttt{while c: s} : S_3}\ \text{S-WhileTrue}$$

`if` runs one branch. The type rule checked both.

`while` uses itself as a premise: that is the loop.

`print` and runtime errors.

$$\frac{S \vdash e : v, S_1}{S \vdash \texttt{print}(e) : S_1 + v}\ \text{S-Print}$$

$$\frac{S \vdash e_1 : v_1, S_1 \qquad S_1 \vdash e_2 : v_2, S_2 \qquad v_2 \neq 0}{S \vdash e_1 \texttt{ // } e_2 : v_1 \div v_2, S_2}\ \text{E-Div}$$

$$\frac{S \vdash e_1 : v_1, S_1 \qquad S_1 \vdash e_2 : 0, S_2}{S \vdash e_1 \texttt{ // } e_2 : \textbf{error}, S_2}\ \text{E-DivZero}$$

`S + v`
the store with one more line of output

`error`
the program stops, exit status 1

A runtime error is a rule, not a crash. The type checker accepts `10 // 0`.

Example: inc(x).

run.py · from Lecture 9

```
x: int = 3
def inc(a: int) -> int:
    return a + 1
print(inc(x) < 5)
```

Before line 4: $S_0 = \{x \mapsto 3\}$ Inside the call: $S_1 = S_0[a \mapsto 3] = \{x \mapsto 3, a \mapsto 3\}$
After return: $S_2 = S_1[\langle ret \rangle \mapsto 4]$

$$\dfrac{\dfrac{}{S_0 \vdash x : 3, S_0}\ \text{E-Var} \qquad \dfrac{\dfrac{\dfrac{}{S_1 \vdash a : 3, S_1}\ \text{E-Var} \qquad \dfrac{}{S_1 \vdash 1 : 1, S_1}\ \text{E-Const}}{S_1 \vdash a + 1 : 4, S_1}\ \text{E-BinOp}}{S_1 \vdash \text{return } a + 1 : S_2}\ \text{S-Return}}{S_0 \vdash \text{inc}(x) : 4, S_0}\ \text{E-Call}$$

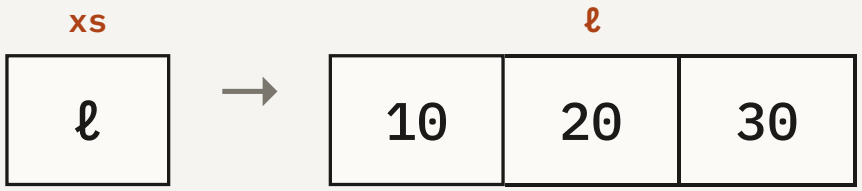
Lecture 9's tree said `int` here. This one says 4. After the call, a and ⟨`ret`⟩ are gone.

Example: the rest of line 4.

$$\dfrac{\dfrac{S_0 \vdash \texttt{inc(x)} : 4, S_0 \quad \text{(slide 9)} \qquad S_0 \vdash 5 : 5, S_0}{S_0 \vdash \texttt{inc(x)} < 5 : \texttt{True}, S_0}\ \text{E-BinOp}}{S_0 \vdash \texttt{print(inc(x)} < 5\texttt{)} : S_0 + \texttt{True}}\ \text{S-Print}$$

The program's meaning is its output: `True`.

A list value is a location.




```
S = { xs ↦ ℓ,     ℓ ↦ [10, 20, 30] }
      variable      location

S(xs) = ℓ                 the value of xs is a location
S(ℓ)  = [10, 20, 30]      the list stored there
```

ℓ

a location: a name for one list. S maps it to the list's elements.

A list variable's value is ℓ, not the elements.

None is a value too: no list at all.

Typing rules.

$$\frac{\Gamma \vdash e_1 : T \quad \ldots \quad \Gamma \vdash e_n : T \quad (n \geq 1)}{\Gamma \vdash [e_1, \ldots, e_n] : [T]}\ \text{[list-display]}$$

$$\frac{\Gamma \vdash e_1 : [T] \qquad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1[e_2] : T}\ \text{[list-select]}$$

$$\frac{\Gamma \vdash e : [T]}{\Gamma \vdash \text{len}(e) : \text{int}}\ \text{[len]}$$

$$\frac{\Gamma \vdash e_1 : [T] \qquad \Gamma \vdash e_2 : \text{int} \qquad \Gamma \vdash e_3 : T}{\Gamma, R \vdash e_1[e_2] = e_3}\ \text{[list-assign-stmt]}$$

`[T]`
a list of Ts, e.g. `[int]`

All elements share one type. The index is an `int`.

[] and None.

```
xs: [int] = []        # ok
xs: [int] = None      # ok: a list may be None
n:  int   = None      # error: expected int, found None
```

the manual

§2.4 writes this as a relation, $\leq_a$. It accepts the same programs.

These two have no type of their own. They take the type of the variable they're stored into.

Evaluation rules.

$$\frac{S \vdash e_1 : v_1, S_1 \quad \dots \quad S_{n-1} \vdash e_n : v_n, S_n \qquad \ell \text{ not in } S_n}{S \vdash [e_1, \dots, e_n] : \ell, S_n[\ell \mapsto [v_1, \dots, v_n]]}\ \text{E-List}$$

$$\frac{S \vdash e_1 : \ell, S_1 \qquad S_1 \vdash e_2 : k, S_2 \qquad S_2(\ell) = [v_0, \dots, v_{n-1}] \qquad 0 \le k < n}{S \vdash e_1[e_2] : v_k, S_2}\ \text{E-Index}$$

$$\frac{S \vdash e : \ell, S_1 \qquad S_1(\ell) = [v_0, \dots, v_{n-1}]}{S \vdash \text{len}(e) : n, S_1}\ \text{E-Len}$$

$$\frac{S \vdash e_1 : \ell, S_1 \qquad S_1 \vdash e_2 : k, S_2 \qquad S_2 \vdash e_3 : v, S_3 \qquad S_3(\ell) = [v_0, \dots, v_{n-1}] \qquad 0 \le k < n}{S \vdash e_1[e_2] = e_3 : S_3[\ell \mapsto [v_0, \dots, v_{k-1}, v, v_{k+1}, \dots, v_{n-1}]]}\ \text{S-ListAssign}$$

ℓ not in S_n

a new location: every list display makes a new list

Indexing reads the list at ℓ. Assignment replaces one element, in place.

Errors for lists.

$$\frac{S \vdash e_1 : \ell, S_1 \quad S_1 \vdash e_2 : k, S_2 \quad S_2(\ell) \text{ has } n \text{ elements} \quad k < 0 \text{ or } k \geq n}{S \vdash e_1[e_2] : \textbf{error}, S_2}\ \text{E-IndexError}$$

$$\frac{S \vdash e_1 : \text{None}, S_1}{S \vdash e_1[e_2] : \textbf{error}, S_1}\ \text{E-IndexNone}$$

(also for len and list assignment)

The type checker can't see either: `i : int` says nothing about its value.

Each has a rule, so each is a defined error.

Example: two names, one list.

alias.py · prints 9

```
xs: [int] = [10, 20]
ys: [int] = xs
ys[0] = 9
print(xs[0])
```

the store after each line

after line	S
1	{xs ↦ ℓ, ℓ ↦ [10, 20]}
2	{xs ↦ ℓ, ys ↦ ℓ, ℓ ↦ [10, 20]}
3	{xs ↦ ℓ, ys ↦ ℓ, ℓ ↦ [9, 20]}

$S_3 = \{xs \mapsto \ell, ys \mapsto \ell, \ell \mapsto [9, 20]\}$

$$\dfrac{\dfrac{}{S_3 \vdash xs : \ell, S_3}\ \text{E-Var} \qquad \dfrac{}{S_3 \vdash 0 : 0, S_3}\ \text{E-Const} \qquad S_3(\ell) = [9, 20]}{S_3 \vdash xs[0] : 9, S_3}\ \text{E-Index}$$

$$\dfrac{S_3 \vdash xs[0] : 9, S_3}{S_3 \vdash \text{print}(xs[0]) : S_3 + 9}\ \text{S-Print}$$

Line 2 copies ℓ, not the list. So changing ys changes xs.

Types meet values

Well-typed programs don’t get stuck.

A run ends one of three ways: a value of the predicted type, a named `error`, or never.

If the program type-checks, no fourth way is possible.

stuck

no rule applies. `4 < True` is stuck: no rule compares an `int` with a `bool`. The type checker rejects it first.

PA4.

PA4, due Fri 11/13: lists and strings. Slides 12–13 are the checker; slides 14–15 say what the output must be.

Thursday’s section is the PA4 lab. No lecture Wed 11/11.

